Chromatic Graph Theory
Circular Coloring of a Graph – Approximation Algorithms
Functional Documentation

Emil Łasocha, Jakub Pączka, Filip Shramko

April 18, 2026

Contents

1 Problem description **2**

2 Solution description – Algorithm 1 **2**
2.1 Algorithm Overview . . . 2
2.2 Pseudocode . . . 3
2.3 Correctness . . . 4
2.4 Accuracy and Quality . . . 4
2.5 Time complexity . . . 5
2.6 Format of an input file and an output file + example . . . 5

3 Solution description – Algorithm 2 **6**
3.1 Algorithm Overview . . . 6
3.2 Pseudocode . . . 7
3.3 Correctness . . . 8
3.4 Accuracy and Quality . . . 8
3.5 Time complexity . . . 9
3.6 Format of an input file and an output file + example . . . 9

4 Solution description – Algorithm 3 **10**
4.1 Algorithm Overview . . . 10
4.2 Pseudocode . . . 11
4.3 Correctness . . . 12
4.4 Accuracy and Quality . . . 12
4.5 Time Complexity . . . 13
4.6 Format of an input file and an output file + example . . . 13

1 Problem description

The goal of this project is to design and implement three approximation algorithms that provide a circular coloring of a graph and find the circular chromatic number of a graph. Let us first provide the necessary definitions.

Def 1. Let $k, d \in \mathbb{N}$, where $k \geqslant d$. We say that a graph $G = (V, E)$ has a **valid** (k, d)**–coloring** if there exists a function $\varphi\colon V \to \{0, 1, ..., k-1\}$ such that for any $v_i v_j \in E$ we have:

$$d \leqslant |\varphi(v_i) - \varphi(v_j)| \leqslant k - d.$$

Def 2. We define the **circular chromatic number** of a graph $G = (V, E)$ as:

$$\chi_c(G) = \inf_{k \geqslant d} \left\{ \frac{k}{d} \colon \text{ graph } G \text{ is } (k, d)\text{-colorable} \right\}.$$

There however exists an alternative, equivalent definition:

Def 3. Let $r > 0$ and let C^r denote a circle of circumference r. A **circular** r**–coloring** of a graph $G = (V, E)$ is a function φ that assigns to every vertex $v \in V$ an open arc $\varphi(v)$ of length 1 on C^r such that for every edge $uv \in E$ the arcs $\varphi(u)$ and $\varphi(v)$ are disjoint.

Equivalently, we may represent each vertex v by a point $\theta_v \in [0, r)$ corresponding to the starting point of the arc $\varphi(v) = (\theta_v, \theta_v + 1)$ on the circle (taken modulo r). For every edge $uv \in E$ the arcs must not intersect, which is equivalent to the condition

$$\mathrm{dist}_r(\theta_u, \theta_v) \geq 1,$$

where

$$\mathrm{dist}_r(a, b) = \min(|a - b|,\, r - |a - b|)$$

denotes the circular distance on the circle C^r.

Under this definition, the circular chromatic number of a graph G can also be expressed as

$$\chi_c(G) = \inf \{ r > 0 \ : \ G \text{ admits a circular } r\text{–coloring} \}.$$

2 Solution description – Algorithm 1

2.1 Algorithm Overview

The motivation behind the first approximation algorithm is to capture the fact that for every positive integer n, we have $\chi_c(C_{2n+1}) = 2 + \frac{1}{n}$, which is a known result[1]. At the same time, the algorithm is designed to provide correct and reasonably accurate results for general graphs.

The main idea is to fix a parameter d and determine the smallest possible value of k for which a valid (k, d)–coloring exists. To achieve this, the algorithm considers all values $d \in \{1, \ldots, D\}$, where D is a specified (by us) upper bound. Reasonable choice of the value D would be $D \approx n$, as it would yield us the correct result for odd cycle graphs. For each fixed d, the algorithm initializes k with the value $2d$ (which is the lowest possible) and then increases it in the process of algorithm, so that the properties of Def 1. hold for this graph.

The algorithm is using a Depth–First Search traversal combined with (mostly) a greedy color assignment strategy. The algorithm's runtime can be high due to the fact that we need to both increase number of possible color (c) assigned to a vertex and parameter k simultanously. It is a balance between increasing the assigned colors and adjusting the parameter k. This is why we cover 4 separate cases (in the `Find-Min-Valid-Color` function below).

Additionally, the reason why it is mostly a greedy coloring is the fact that it prevents assigning identical colors to the a vertex v and the parent of the parent of vertex v. This small change was needed to correctly handle odd cycles C_{2n+1}. It helps assigning more valid colors and not reuse the same ones (which is useful especially for $d \geqslant 2$).

[1] *Modele i metody kolorowania grafów* s.151, Adam Nadolski pod redakcją Marka Kubale, 2002r.

2.2 Pseudocode

Algorithm 1 DFS–based Approximation Algorithm

Inputs: Graph $G = (V, E)$, maximum parameter D

Outputs: Approximation of $\chi_c(G)$ and a valid circular coloring of graph G

```
if |E| = 0 then
    return χ_c(G) = 1, finalcoloring = [0, 0, ..., 0]
end if
Initialize χ_C(G) ← ∞ and finalcoloring[v] ← UNCOLORED for all v ∈ V
Initialize k[value] ← 2 · value for value ∈ {1, 2, ..., D}
for d = 1, 2, ..., D do
    Initialize:
        colors[v] ← UNCOLORED for all v ∈ V
        visited[v] ← False for all v ∈ V
        parent[v] ← NULL for all v ∈ V
    while there exists v such that colors[v] = UNCOLORED do
        Choose v_0 with maximum degree from those v where colors[v] = UNCOLORED
        colors[v_0] ← 0
        DFS-Color(v_0)
    end while
    ratio[d] ← k[d]/d
    if ratio[d] < χ_C(G) then
        χ_C(G) ← ratio[d]
        finalcoloring[v] ← colors[v] for all v ∈ V
    end if
end for
return χ_C(G), finalcoloring
```

Function DFS-Color(v)

```
visited[v] ← True
for each neighbor u of v do
    if visited[u] = False then
        parent[u] ← v
        c ← Find-Min-Valid-Color(u, v)
        colors[u] ← c
        DFS-Color(u)
    end if
end for
```

Function Find-Min-Valid-Color(u, p)

```
c ← 0
k_lowest ← k[d]
while  do
    valid ← True
    for each w ∈ N(u) with colors[w] ≠ UNCOLORED do
        diff = c − colors[w]
        if |diff| < d then
            valid ← False
            break
        else if diff > k_lowest − d then
```

```
11:             valid ← False
12:             k_lowest ← k_lowest + 1
13:             c ← c − 1
14:             break
15:         else if −diff > k_lowest − d and c < d then
16:             valid ← False
17:             break
18:         else if −diff > k_lowest − d and c ⩾ d then
19:             valid ← False
20:             k_lowest ← k_lowest + 1
21:             break
22:         end if
23:     end for
24:     if valid and parent[p] ≠ NULL then
25:         if c = colors[parent[p]] then
26:             valid ← False
27:         end if
28:     end if
29:     if valid then
30:         k[d] ← max{k_lowest, c + 1}
31:         return c
32:     end if
33:     c ← c + 1
34: end while
```

2.3 Correctness

We have two while loops, which we want to ensure that terminate.

In line 11 of **Algorithm 1** we have a while loop.
In the case of disconnected graphs, the `while` loop with each step colors one connected component. For any connected component and after every `while` step, either all its vertices are colored, or all its vertices are uncolored.

In line 3 of `Find-Min-Valid-Color` we have a while loop. We will prove that the while loop eventually terminates.
– If $|\text{diff}| < d$ (line 7): after the break, we increment c, so diff increases, therefore it will happen that $\text{diff} \geqslant d$ in some time.
– If $\text{diff} > k_{\text{lowest}} - d$ (line 10): c stays in place, k rises, so eventually $\text{diff} \leqslant k_{\text{lowest}} - d$.
– If $-\text{diff} > k_{\text{lowest}} - d$ and $c < d$ (line 15): then c increments, so eventually will be $c \geqslant d$.
– If $-\text{diff} > k_{\text{lowest}} - d$ and $c \geqslant d$ (line 18): then both k and c increases, so $-\text{diff}$ decreases and $k_{\text{lowest}} - d$ increases, so eventually will be $-\text{diff} \leqslant k_{\text{lowest}} - d$.
– No two of the above can occure at the same time since $d \leqslant k_{\text{lowest}} - d$.

2.4 Accuracy and Quality

Let's check it on five graphs: C_5, C_6, K_3, K_4 and path P_6. Let's set $D = 3$. The optimal results are respectively[1]: $\frac{5}{2}, 2, 3, 4, 2$.

- As desired, for graph C_5 and $d = 2$ we have the coloring $[0, 2, 4, 1, 3]$, giving us the correct result $\chi_c(C_5) = \frac{5}{2}$.

- For C_6 and $d = 1$ we have the coloring $[0, 1, 2, 0, 1, 2]$, so the algorithm suggests the answer is 3, while it is not, because $\chi_c(C_6) = 2$.
- For K_3 and $d = 1$ we have the coloring $[0, 1, 2]$, so the algorithm correctly identifies the answer $\chi_c(K_3) = 3$.
- For K_4 and $d = 1$ we have the coloring $[0, 1, 2, 3]$, so the algorithm correctly identifies the answer $\chi_c(K_4) = 4$.
- For P_6 and $d = 3$ we have the coloring $[3, 0, 3, 6, 2, 5]$, so we got $\frac{7}{3}$, but the answer is $\chi_c(P_6) = 2$. But the difference is only $\frac{1}{3}$ which is worth mentioning.

Overall the algorithm performs quite well.

2.5 Time complexity

Let $n = |V|$ and $m = |E|$.

The algorithm consists of three main parts: the outer loop over parameter d, the DFS traversal of the graph, and the color-selection procedure.

The outer loop iterates over all values $d \in \{1, ..., D\}$, giving a factor of D.

For each fixed d, the algorithm performs a DFS traversal. This traversal ensures that each vertex and each edge is processed at most once, giving a complexity of $O(n+m)$. But the main cost is from the color selection function during the traversal.

For a fixed vertex, the algorithm tests candidate colors incrementally (c increments in three cases), giving the $O(n)$ complexity. But, in the last, worst case (when diff $> k_{\text{lowest}} - d$), the value of c doesn't increment – the value k increments. The algorithm may require $k = O(D \cdot n)$ colors. So overall it is $O(D \cdot n)$ for a fixed vertex.

For each candidate color, the algorithm checks all already colored neighbors of the vertex, which takes $O(\deg(v))$ time. Summing over all vertices, testing a single candidate color across all vertices costs $O(m)$. Since each vertex tests $O(D \cdot n)$ candidate colors in the worst case, the total cost of the color-selection for a single value of d is $O(D \cdot n \cdot m)$.

Combining all components (and keeping in mind that DFS traversal is not nested, but sequential), the overall worst-case time complexity of the algorithm is

$$O(D^2 \cdot n \cdot m).$$

In dense graphs, where $m = O(n^2)$, this becomes $O(D^2 \cdot n^3)$.

2.6 Format of an input file and an output file + example

Input:
— The first line contains two integers: n – the number of vertices and m – the number of edges.
— Each of the following m lines contains of two integers i and j separated by a space, where $i, j \in \{0, 1 \ldots n-1\}$. Each line represents an edge ij in the input graph.
— The last line contains an integer D, which denotes the parameter D in the algorithm.
Output:
— The first line contains two integers: k and d separated by a space, denoting that the algorithm has found a (k, d)–coloring of the graph.
— The next line contains n integers representing the colors assigned to the vertices $0, 1, \ldots, n-1$.
Example Input File:
3 3
1 2
0 2
0 1

5
Example Output File:
3 1
0 1 2

3 Solution description – Algorithm 2

3.1 Algorithm Overview

The second approximation algorithm is based on the geometric interpretation of circular coloring using arcs on a circle (see Def 3.). Instead of directly searching for integers k and d, the algorithm operates on the equivalent definition where vertices are assigned arcs of length 1 on a circle of circumference r.

More precisely, each vertex $v \in V$ is represented by a point $\theta_v \in [0, r)$ which corresponds to the starting point of an open arc of length 1 on the circle. For every edge $uv \in E$, the arcs assigned to u and v must be disjoint, which means that the circular distance between θ_u and θ_v on the circle of length r is at least 1.

The algorithm searches for the smallest value of r for which such a placement of vertices exists. This is done using a binary search over real values of r. The search range is initialized to $[1, |V|]$, since the circular chromatic number of any graph satisfies $1 \leq \chi_c(G) \leq |V|$.

For each candidate value of r, the algorithm attempts to construct a feasible placement of vertices on the circle. Initially, vertices are placed evenly around the circle. Then an iterative correction procedure is applied. During this procedure, every edge that violates the distance constraint generates a corrective force that pushes its endpoints apart along the circle.

These forces are accumulated during each iteration and applied simultaneously at the end of the iteration. After a given number T of iterations, the algorithm checks whether all edges satisfy the required distance constraint. If the placement is feasible, the binary search continues on the lower half of the interval in search for a better solution, otherwise it continues on the upper half.

3.2 Pseudocode

Algorithm 2 Circle Relaxation Approximation Algorithm

Inputs: Graph $G = (V, E)$, binary search steps B, iterations T
Outputs: Approximation of $\chi_c(G)$ and a circular coloring of G

```
if |E| = 0 then
    return χc(G) = 1, finalcoloring = [0, 0, ..., 0]
end if
lower ← 1
upper ← |V|
bestAngles ← INITIALIZE-POSITIONS(G, |V|)
for i = 1, 2, ..., B do
    r ← (lower + upper)/2
    θ ← INITIALIZE-POSITIONS(G, r)
    θ ← RELAXATION-SOLVER(G, θ, r, T)
    if CHECK-FEASIBLE(G, θ, r) then
        upper ← r
        bestAngles ← θ
    else
        lower ← r
    end if
end for
return upper, bestAngles
```

Function Initialize-Positions(G, r)

```
n ← |V|
for each vertex v_i ∈ V do
    θ[v_i] ← i · r/n
end for
return θ
```

Function Relaxation-Solver(G, θ, r, T)

```
for i = 1, 2, ..., T do
    for each v ∈ V do
        shiftSum[v] ← 0
    end for
    for each edge (u, v) ∈ E do
        Δ ← θ[u] − θ[v]
        Wrap Δ into [−r/2, r/2)
        dist ← |Δ|
        violation ← 1 − dist
        if violation > 0 then
            shift ← violation/2
            if Δ > 0 then
                shiftSum[u] ← shiftSum[u] + shift
                shiftSum[v] ← shiftSum[v] − shift
            else
                shiftSum[u] ← shiftSum[u] − shift
                shiftSum[v] ← shiftSum[v] + shift
```

```
            end if
        end if
    end for
    for each v ∈ V do
        θ[v] ← θ[v] + α · shiftSum[v]        ▷ We multiply the shift by some constant α
        Wrap θ[v] into [0, r)
    end for
end for
return θ
```

Function Check-Feasible(G, θ, r)

```
for each edge (u, v) ∈ E do
    Δ ← θ[u] − θ[v]
    Wrap Δ into [−r/2, r/2)
    dist ← |Δ|
    if dist < 1 then
        return False
    end if
end for
return True
```

3.3 Correctness

We will show that the algorithm terminates and returns a correct circular r-coloring for some value r.

First, observe that the outer loop of the algorithm performs a fixed number B of iterations of binary search. Since B is a value chosen beforehand, this loop always terminates.

For every value of r considered during the binary search, the algorithm attempts to construct a feasible placement of vertices on the circle. This is done using the procedure `Relaxation-Solver`, which performs exactly T iterations. Since T is also a value chosen beforehand, this procedure always terminates.

After the relaxation process finishes, the algorithm verifies whether the obtained placement satisfies the constraint

$$\mathrm{dist}_r(\theta_u, \theta_v) \geq 1$$

for every edge $uv \in E$. If this condition holds, the placement defines a valid circular r–coloring of the graph, since the corresponding arcs of length 1 on the circle C^r are disjoint. Therefore any circular r–coloring returned by the algorithm will be valid.

Note that the algorithm will always return some valid circular r-coloring, as the initial values of $upper \leftarrow |V|$ and $bestAngles \leftarrow$ Initialize-Positions$(G, |V|)$ (which corresponds to assigning equidistant points on a circle of radius $|V|$ to each vertex) describe a valid circular $|V|$-coloring for any graph G.

3.4 Accuracy and Quality

The quality of the approximation depends on the number of binary search steps B and the number of relaxation iterations T.

Due to its nature of finding a real-valued solution it is difficult to precisely judge the algorithm's quality before implementation. It depends on several factors including the initial order of vertex placement on the circle and the choice of the B and T parameters, for which the optimal values will have to be determined experimentally.

It can be noted, that the algorithm will always find the optimal solution for K_n full graphs. Their optimal r–coloring is equivalent to the equidistant placement of vertices on the circle performed at the beginning of the algorithm's runtime, which will not be able to be corrected by the algorithm.

3.5 Time complexity

The algorithm consists of two main components: the binary search over the parameter r and the relaxation procedure used to test feasibility.

The outer loop performs B iterations of binary search. This contributes a factor of B to the running time.

For each value of r, the algorithm runs the `Relaxation-Solver` procedure for T iterations. During every iteration, each edge of the graph is processed exactly once in order to check whether the distance constraint between its endpoints is satisfied. This requires $O(|E|)$ time.

After the relaxation phase, the algorithm performs a feasibility check that also scans all edges, which takes an additional $O(|E|)$ time.

Therefore, the cost of a single feasibility test is

$$O(T \cdot |E|).$$

Combining this with the binary search loop, the overall time complexity of the algorithm is

$$O(B \cdot T \cdot |E|).$$

3.6 Format of an input file and an output file + example

Input:
— The first line contains two integers: n – the number of vertices and m – the number of edges.
— Each of the following m lines contains of two integers i and j separated by a space, where $i, j \in \{0, 1 \ldots n-1\}$. Each line represents an edge ij in the input graph.
— The last line contains two positive integers T and B, where B is the number of binary search steps and T is the number of relaxation iterations.
Output:
— The first line contains a floating–point number r, denoting that the algorithm has found an r–circular coloring of the input graph.
— The next line contains n floating–point numbers representing the colors assigned to the vertices $0, 1, \ldots, n-1$.
Example Input File:
4 4
1 2
0 2
0 1
0 3
40 500
Example Output File:
3.02
0.02 1.03 2.04 1.02

4 Solution description – Algorithm 3

4.1 Algorithm Overview

The main idea of the third algorithm is to fix two parameters k and d, where k is bounded by a predefined value K, and to determine whether a valid (k, d)–coloring exists. To achieve this, the algorithm considers all pairs (k, d) such that $k \in \{1, \ldots, K\}$ and $d \in \{1, \ldots, k\}$. For each such pair, it attempts to construct a valid coloring satisfying the constraints of Def 1.. Among all successful attempts, the algorithm returns the smallest ratio k/d, which corresponds to the circular chromatic number. Reasonable choice of the value K would be $K \approx n$ for a dense graph, as it would yield us the correct result for complete graphs. For a sparse graph, consider $K \approx \sqrt{n}$ for the faster output.

The algorithm uses BFS combined with a greedy color assignment strategy. It maintains a `frontier` set of vertices, which represents the current "layer" of exploration. Starting from an arbitrary uncolored vertex, the algorithm assigns it color 0 and iteratively colors its neighbors in a greedy way – the smallest possible number that fulfills all inequalities. Then neighbors of neighbors, and so on. This process continues until no further vertices can be colored within the current connected component. This is repeated in case of the graph consists of multiple connected components.

4.2 Pseudocode

Algorithm 3 Greedy–BFS (k, d)–fixed Algorithm

Inputs: Graph $G = (V, E)$, fixed integer K
Outputs: Circular chromatic number $\chi_c(G)$ and a valid coloring

```
if |E| = 0 then
    return χc(G) = 1, finalcoloring = [0, 0, ..., 0]
end if
Initialize χc(G) ← ∞
Initialize finalcoloring[v] ← UNCOLORED for all v ∈ V
for k = 1, 2, ..., K do
    for d = 1, 2, ..., k do
        Initialize:
          colors[v] ← UNCOLORED for all v ∈ V
          valid ← True
        while there exists v such that colors[v] = UNCOLORED and valid do
            Choose arbitrary uncolored vertex v1
            colors[v1] ← 0
            frontier ← {v1}
            while frontier ≠ ∅ and valid do
                new_frontier ← ∅
                for each v ∈ frontier do
                    for each neighbor u ∈ N(v) do
                        if colors[u] = UNCOLORED then
                            c ← Find-Min-Valid-Color(u, d, k)
                            if c = FAIL then
                                valid ← False
                                break
                            end if
                            colors[u] ← c
                            add u to new_frontier
                        end if
                    end for
                end for
                frontier ← new_frontier
            end while
        end while
        if valid then
            ratio ← k/d
            if ratio < χc(G) then
                χc(G) ← ratio
                finalcoloring[v] ← colors[v] for all v ∈ V
            end if
        end if
    end for
end for
return χc(G), finalcoloring
```

Function Find-Min-Valid-Color(u, d, k)

```
1: c ← 0
2: while True do
3:     valid ← True
4:     for each w ∈ N(u) with colors[w] ≠ UNCOLORED do
5:         absdiff ← |colors[w] − c|
6:         if absdiff < d or absdiff > k − d then
7:             valid ← False
8:             break
9:         end if
10:    end for
11:    if valid then
12:        return c
13:    end if
14:    c ← c + 1
15:    if c ⩾ k then
16:        return FAIL
17:    end if
18: end while
```

4.3 Correctness

In line 11 of **Algorithm 3** we have a `while` loop.
In the case of disconnected graphs, the `while` loop with each step colors one connected component. For any connected component and after every `while` step, either all its vertices are colored, or all its vertices are uncolored.

In line 15 of **Algorithm 3** the `while` loop over the frontier also terminates. At each step, vertices from the current frontier are used to color previously uncolored neighbors, which are then added to the new frontier. The frontier strictly progresses through the graph and eventually becomes empty, so the loop terminates.

In line 3 of `Find-Min-Valid-Color` we have a `while` loop. At each iteration, the algorithm checks whether the current color c satisfies the inequalities for all already colored neighbors. If not, c is incremented. Since c increases and is bounded above by k, the loop always terminates.

We can see that all while loops are guaranteed to terminate, and therefore the algorithm will always terminate. Due to the coloring approach, where we only color vertices with colors available to them, any coloring returned by the algorithm will be valid.

4.4 Accuracy and Quality

Let's check it on five graphs: C_5, C_6, K_3, K_4 and path P_6. Let's set $K = 4$. The optimal results are respectively[1]: $\frac{5}{2}, 2, 3, 4, 2$.

- As desired, for graph C_5 we have the coloring $[0, 1, 0, 1, 2]$, giving us the incorrect result $\chi_c(C_5) = 3$.
- For C_6 we have the correct coloring $[0, 1, 0, 1, 0, 1]$ and $\chi_c(C_6) = 2$.
- For K_3 we have the coloring $[0, 1, 2]$, so the algorithm correctly identifies the answer $\chi_c(K_3) = 3$.
- For K_4 we have the coloring $[0, 1, 2, 3]$, so the algorithm correctly identifies the answer $\chi_c(K_4) = 4$.

- For P_6 we have the coloring $[0, 1, 0, 1, 0, 1]$, we have the correct coloring and $\chi_c(P_6) = 2$.

Overall the algorithm performs quite well, but can do worse for large graphs.

4.5 Time Complexity

Let $n = |V|$ and $m = |E|$.
The algorithm iterates over all pairs (k, d) such that $k \in \{1, \ldots, K\}$ and $d \in \{1, \ldots, k\}$. Therefore, the total number of iterations is

$$\sum_{k=1}^{K} k = \frac{K(K+1)}{2} = O(K^2).$$

For each fixed pair (k, d), the algorithm performs a traversal of the graph using the frontier-based approach. Each vertex is colored at most once, and each edge is considered at most a constant number of times when exploring neighbors. This part takes $O(n + m)$ time.
For each vertex u, the function `Find-Min-Valid-Color` is called once. In the worst case, it tries all colors $c \in \{0, 1, \ldots, k-1\}$, which gives $O(k)$ iterations. For each candidate color, all already colored neighbors of u are checked, which takes $O(\deg(u))$ time. Thus, the cost per vertex is $O(k \cdot \deg(u))$, and summing over all vertices gives $O(k \cdot m)$.
Combining the above (since BFS is not nested), for each pair (k, d) the algorithm runs in $O(K \cdot m)$ time. Summing over all pairs (k, d), we obtain $O(K^3 \cdot m)$.
In dense graphs, where $m = O(n^2)$, this becomes $O(K^3 \cdot n^2)$

4.6 Format of an input file and an output file + example

Input:
— The first line contains two integers: n – the number of vertices and m – the number of edges.
— Each of the following m lines contains of two integers i and j separated by a space, where $i, j \in \{0, 1 \ldots n-1\}$. Each line represents an edge ij in the input graph.
— The last line contains an integer K, which denotes the parameter K in the algorithm.
Output:
— The first line contains two integers: k and d separated by a space, denoting that the algorithm has found a (k, d)–coloring of the graph.
— The next line contains n integers representing the colors assigned to the vertices $0, 1, \ldots, n-1$.
Example Input File:
3 3
1 2
0 2
0 1
4
Example Output File:
3 1
0 1 2